Training Scope is a Hidden Variable

A Factorial Study of Neural Post-Processing for JPEG Compression

Mariya Derrag
FST Mohammedia, Universite Hassan II, Casablanca, Morocco
derragmaria@gmail.com

May 2026

Abstract

Neural post-processing for JPEG compression is typically evaluated architecture by architecture. Training scope—specialist per quality level or generalist global—gets treated as an implementation detail. It is not.

A factorial study crosses three architectures (DnCNN, FFDNet, ARCNN) with two training scopes. Results are non-monotonic. DnCNN global beats the specialist at Q=50 (+0.43 dB). FFDNet degrades under both scopes even with per-image adaptive σ, pointing to a structural mismatch between scalar conditioning and block-structured JPEG artifacts. ARCNN global exhibits pathological residual kurtosis runaway at high quality ($\kappa-3$ up to +6.3): the model hallucinates structure where none remains. None of these effects appear when scope is not treated as an independent variable.

A from-scratch corrected JPEG codec reaches parity with PIL `optimize=True`, isolating the neural contribution from codec implementation artifacts.

1 Introduction

The standard protocol: pick an architecture, train it on compressed images, report PSNR. Training scope does not appear in the results table. It should.

A 3×2 factorial design shows that the scope×quality interaction is non-monotonic and architecture-dependent. DnCNN global wins at Q=50 (+0.43 dB). FFDNet loses under both regimes even with adaptive σ. ARCNN global drifts in kurtosis at Q=90 ($\kappa-3 = +6.3$). These effects are invisible when scope varies only within a single cell.

The practical implication is direct. The field's assumption that per-quality specialization is always optimal is false for at least one major architecture at one major quality level. The corollary—that global training is always worse—is also false: ARCNN global wins marginally at Q=75. What works depends on the architecture and target quality in ways that are not predictable without systematic comparison.

Contributions. (1) A factorial benchmark of six cells (3 architectures × 2 scopes) with a full reproducibility specification. (2) The first report, to our knowledge, of a non-monotonic scope×quality effect in JPEG post-processing: DnCNN global beats the specialist at Q=50. (3) Evidence that FFDNet's degradation on JPEG is structural rather than protocolary, persisting under per-image adaptive σ and per-quality training. (4) A residual kurtosis diagnostic that distinguishes honest correction from pathological over-correction.

2 Controlled Baseline: A Corrected JPEG Codec

Before measuring a neural contribution, we need to know what the codec alone produces. We build a JPEG codec from scratch, identify three implementation errors, and show that the corrected version reaches parity with optimized industrial encoders.

2.1 Three Implementation Errors

A naive Python reimplementation of JPEG can fall 10 dB behind `libjpeg` without a single line being obviously wrong. Three independent bugs account for the gap.

F1. Quantization table scaling. The linear approximation $s = 100 - Q$ underestimates the step at low Q. The correct IJG formula is piecewise (Eq. 1).

F2. Premature EOB insertion (dominant). The encoder placed an end-of-block marker after five consecutive zero AC coefficients, silently dropping all non-zero coefficients beyond. Every block was corrupted regardless of Q. Removing this single error raises PSNR by 6–8 dB at Q=10.

F3. Static Huffman tables. Per-image adaptive tables reduce code length by 7–12% for $Q \leq 25$, where AC histograms deviate most from Annex K defaults.

$$s = \begin{cases} 200 - 2Q, & Q \geq 50, \\ 5000 \,/\, Q, & Q < 50. \end{cases} \tag{1}$$

2.2 Baseline Comparison

Table 1: Corrected codec vs. baselines at matched BPP. Δ relative to our codec.

Q	BPP	`libjpeg`	PIL opt.	ΔPIL
10	0.307	22.25 dB	24.66 dB	−1.10 dB
25	0.631	26.97 dB	28.32 dB	−0.48 dB
50	1.018	30.12 dB	30.68 dB	−0.32 dB
75	1.551	32.64 dB	33.01 dB	−0.15 dB
90	2.591	36.12 dB	36.30 dB	+**0.17** dB

Against default `libjpeg`, the corrected codec gains +3.51 dB at Q=10, mostly from adaptive Huffman coding. The gap narrows against PIL optimized and reverses at Q=90. The corrected codec is not fundamentally superior to a well-configured encoder. What it demonstrates is that implementation details dominate rate-distortion performance. This baseline is used for all neural evaluations.

3 Factorial Study: Architecture × Scope

3.1 Design

Six cells: three architectures crossed with two training scopes.

Architectures:

- **DnCNN** [3]: 17-layer residual CNN. Predicts the noise residual; the skip connection allows near-zero correction when the input is already clean.
- **FFDNet** [4]: extends DnCNN with a spatially uniform σ map and subpixel downsampling (2×2 patches rearranged into 4 channels at half resolution). Direct reconstruction, no residual skip.
- **ARCNN** [2]: 4-layer CNN for JPEG artifacts (filters 9×9, 7×7, 1×1, 5×5). Direct reconstruction, trained on JPEG pairs rather than Gaussian noise.

Scopes:

- **Per-quality (PQ):** five independent models, one per $Q \in \{10, 25, 50, 75, 90\}$.
- **Global (GL):** one model trained on all quality levels mixed together.

3.2 Reproducible Specification

Data. DIV2K (800 images), center-cropped to 256×256. Split 70/15/15%, fixed seed 42. Compressed with the corrected DCT codec. Patch extraction: 40 random 64×64 patches per image, with random rotation (0–3), horizontal flip, vertical flip.

PQ protocol:

```python
for Q in [10, 25, 50, 75, 90]:
    train = extract_patches(train_images, Q)
    val = extract_patches(val_images, Q)
    model = Architecture()  # DnCNN or ARCNN
    train_from_scratch(model, train, val, epochs=50)
    save(f"{arch}_pq_q{Q}.pt")
```

Listing 1: PQ training

GL protocol:

```python
train = []
for Q in [10, 25, 50, 75, 90]:
    train += extract_patches(train_images, Q)
model = Architecture()
train_from_scratch(model, train, val_all_Q, epochs=50)
save(f"{arch}_gl.pt")
```

Listing 2: GL training

FFDNet adaptive σ estimation:

```python
def estimate_sigma(Y_comp, Y_orig):
    return np.std(Y_comp - Y_orig) / 255.0

sigma = estimate_sigma(compressed, original)
sigma_map = torch.full((1, 1, H//2, W//2), sigma)
output = ffdnet(input, sigma_map)
```

Listing 3: Adaptive sigma

Table 2: Training hyperparameters.

Parameter	Value
Optimizer	Adam
Learning rate	10^{-3} (DnCNN, FFDNet); 10^{-4} (ARCNN)
Weight decay	10^{-4}
Scheduler	Cosine annealing, $T_{\max} = 50$
Batch size	64 (CUDA), 16 (CPU)
Max epochs	50
Early stopping	5 epochs patience
Gradient clip	Max norm 1.0
Mixed precision	AMP
Loss	MSE

3.3 Results

3.4 Rate-Distortion Curves

Three distinct clouds emerge. DnCNN and ARCNN form a tight upper group. FFDNet-GL sits 5–6 dB lower. FFDNet-PQ forms a third, non-monotonic cluster with a dip at Q=75.

Table 3: PSNR (dB) and ΔPSNR vs. corrected codec. Best per quality bolded; degradation below −0.5 dB marked †. PQ = per-quality; GL = global.

	$Q = 10$		$Q = 25$		$Q = 50$		$Q = 75$		$Q = 90$	
Cell	PSNR	Δ	PSNR	Δ	PSNR	Δ	PSNR	Δ	PSNR	Δ
Baseline	25.52	—	28.52	—	30.64	—	32.81	—	35.84	—
DnCNN-PQ	26.12	+0.60	29.25	+0.73	30.63	−0.01	33.35	+0.55	35.84	≈0
DnCNN-GL	25.97	+0.45	29.01	+0.48	**31.06**	**+0.42**	33.07	+0.26	35.74	−0.09
FFDNet-PQ	21.98	−3.54†	24.46	−4.06†	27.69	−2.95†	25.95	−6.86†	28.52	−7.32†
FFDNet-GL	25.58	+0.06	27.85	−0.67	28.91	−1.73†	29.55	−3.26†	30.06	−5.78†
ARCNN-PQ	**26.06**	**+0.54**	**28.91**	**+0.38**	**30.98**	**+0.34**	32.83	+0.02	35.56	−0.28
ARCNN-GL	25.81	+0.29	28.88	+0.36	31.00	+0.35	**32.93**	**+0.13**	35.10	−0.74†

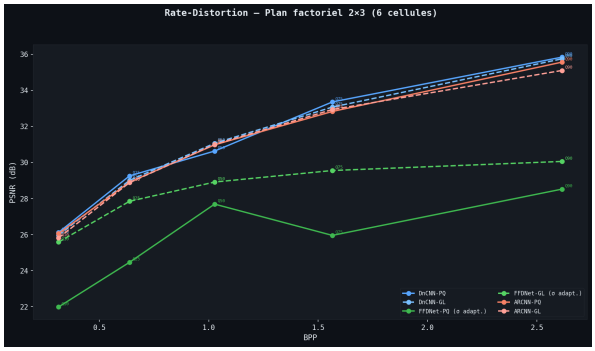

Figure 1: Rate-distortion curves. DnCNN and ARCNN form a tight upper cloud; FFDNet-GL is separated by 5–6 dB; FFDNet-PQ is non-monotonic with a dip at Q=75.

3.5 Scope Effect: Global vs. Per-Quality

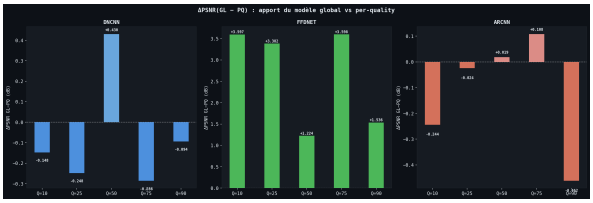

Figure 2: Scope comparison. Positive bars favor global training; negative bars favor specialists.

DnCNN. Non-monotonic. The specialist wins at the extremes (Q=10, Q=75); the global model wins at Q=50 (+0.43 dB). Cross-quality exposure helps when artifacts are heterogeneous.

FFDNet. GL always beats PQ, but both lose. Global training dilutes catastrophic errors; the specialist overfits to one quality level's artifact pattern.

ARCNN. Stable except at Q=90, where PQ wins (+0.46 dB). The specialist knows Q=90 is near-lossless; the global model applies an average correction that destroys fine detail.

3.6 Residual Kurtosis Diagnostic

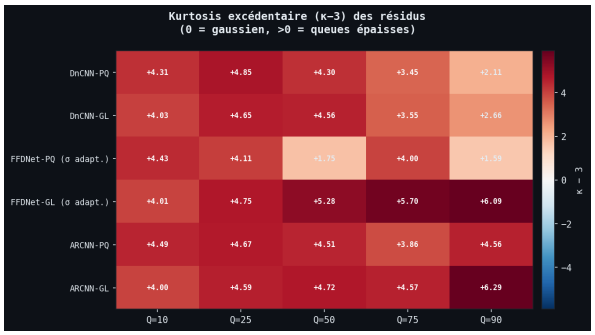

Figure 3: Excess kurtosis of residuals. 0 = Gaussian; > 0 = heavy tails; rising $\kappa-3$ at high quality signals pathological over-correction.

Table 4: Excess kurtosis ($\kappa-3$) of residuals.

Cell	10	25	50	75	90
DnCNN-PQ	+4.31	+4.85	+4.30	+3.45	+2.11
DnCNN-GL	+4.03	+4.65	+4.56	+3.55	+2.66
FFDNet-PQ	+4.43	+4.11	+1.75	+4.00	+1.59
FFDNet-GL	+4.01	+4.75	+5.28	+5.70	+6.09
ARCNN-PQ	+4.49	+4.67	+4.51	+3.86	+4.56
ARCNN-GL	+4.00	+4.59	+4.72	+4.57	+6.29

DnCNN. Honest decay (+4.3 → +2.1): correction magnitude decreases with quality, as expected.

FFDNet-GL and ARCNN-GL. Pathological rise at Q=90 (+6.1 and +6.3). The global models hallucinate structure where none remains.

FFDNet-PQ. Anomalous bimodal behavior (+1.75 at Q=50): a miscalibrated σ over-smooths some regions and under-corrects others.

3.7 Correlation Analysis

Table 5: Pearson correlation (all qualities pooled). $^*p<0.05$.

Cell	$r(\sigma_r, \Delta\text{PSNR})$	$r(\kappa, \Delta\text{PSNR})$
DnCNN-PQ	+0.261	−0.074
DnCNN-GL	+0.204	−0.005
FFDNet-PQ	+0.252*	−0.224
FFDNet-GL	−0.350	−0.107
ARCNN-PQ	−0.268	−0.041
ARCNN-GL	−0.143	−0.020

σ_r predicts gain only for models that can actually help, DnCNN first. κ does not predict gain universally; its trajectory across quality levels is what detects pathology.

4 Discussion

Training scope is a hidden variable. The field evaluates architectures as if scope did not matter. That misses the point. DnCNN global wins at Q=50, ARCNN specialist wins at Q=90, FFDNet loses in both configurations. These interactions do not exist in results tables from papers that never vary scope. The recommendation is simple: future benchmarks should declare scope as an independent variable.

FFDNet: structural mismatch hypothesis. Earlier work attributed FFDNet's degradation to a poorly chosen fixed σ. This study refutes that: with per-image adaptive σ and per-quality training, degradation still reaches −7.32 dB at Q=90. The hypothesis is that FFDNet's scalar σ conditioning and subpixel downsampling are fundamentally mismatched to JPEG's heteroscedastic, block-structured artifacts. A scalar σ cannot represent multi-scale error structure. The downsampling destroys the 8×8 block-grid phase. A full-resolution FFDNet variant without downsampling would test this directly.

ARCNN: domain fit versus skip connection. ARCNN-PQ is the most consistent through Q=50, confirming that JPEG-domain training beats Gaussian pretraining for structured artifacts. At Q=90, both variants degrade, GL more than PQ (−0.74 dB vs. −0.28 dB). The absent residual skip connection denies the model the identity mapping it needs when the codec already produces a high-quality image. An ARCNN variant with a skip connection, trained identically, would resolve this attribution.

Kurtosis as diagnostic, not predictor. The trajectory of $\kappa-3$ across quality levels distinguishes honest correction (DnCNN: decay) from pathological over-correction (GL models: runaway at Q=90). This is not a universal predictor of gain. It is a model-specific pathology detector.

5 Limitations

Dataset. Natural photographs only (DIV2K). Medical and satellite images may behave differently.

FFDNet hypothesis. The structural mismatch attribution (downsampling, scalar σ) is a hypothesis, not a measured result. A full-resolution FFDNet ablation is needed.

ARCNN hypothesis. The skip-connection attribution is similarly untested experimentally.

Kurtosis. The diagnostic value of $\kappa-3$ is established post-hoc; predictive validation (computing κ before model selection) remains future work.

Modern architectures. Transformer-based denoisers (SwinIR, Restormer) are not evaluated.

6 Conclusion

Treating training scope as an implementation detail leads to incomplete conclusions. Six cells (3 architectures × 2 scopes) are enough to show it. The optimal scope varies non-monotonically with architecture and quality level. DnCNN global beats the specialist at Q=50 (+0.43 dB). FFDNet degrades under both regimes, even with adaptive σ. ARCNN global drifts in kurtosis at high quality ($\kappa-3$ up to +6.3).

None of these effects are visible when scope is not a variable. Future benchmarks should explicitly cross architecture with training scope, and report residual kurtosis trajectories as a standard diagnostic.

Acknowledgements

The author thanks Pr. Anouar Ben Loghfyry (FST Mohammedia) for supervision and critical guidance. This work was carried out within the Licence Sciences de Données et Informatique Décisionnelle program, 2025–2026.

References

[1] Agustsson, E., & Timofte, R. (2017). NTIRE 2017 challenge on single image super-resolution: Dataset and study. *IEEE/CVF CVPRW*, 1122–1131.

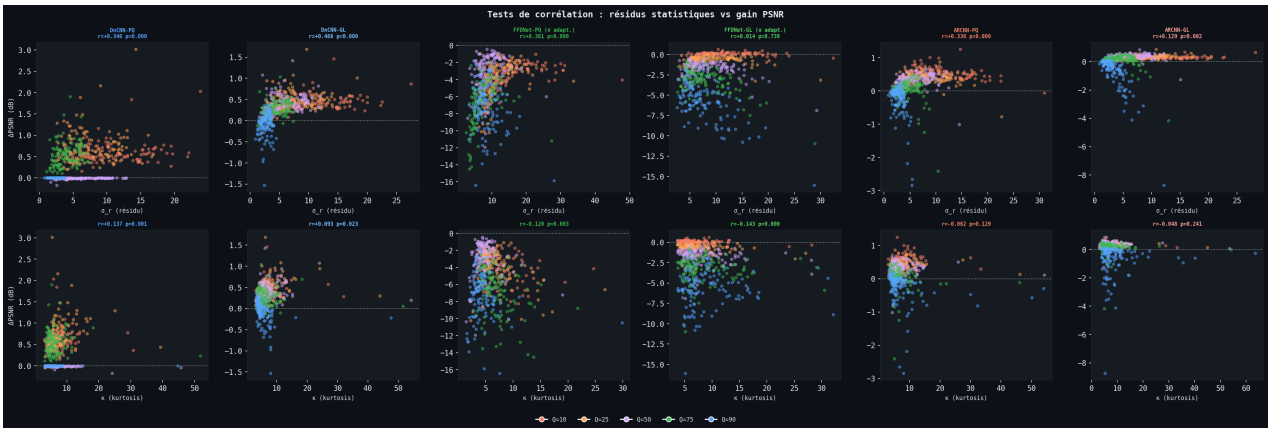

Figure 4: Correlation tests. Colors denote quality levels.

[2] Dong, C., Deng, Y., Loy, C. C., & Tang, X. (2015). Compression artifacts reduction by a deep convolutional network. *ICCV*, 576–584.

[3] Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017). Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising. *IEEE TIP*, 26(7), 3142–3155.

[4] Zhang, K., Zuo, W., & Zhang, L. (2018). FFDNet: Toward a fast and flexible solution for CNN-based image denoising. *IEEE TIP*, 27(9), 4608–4622.